OPERATION KIVOTOS · 사전작업 1단계

몬스터 컴포넌트 확립 — 생애 계약 명세서

풀에서 재사용되는 몬스터가 **매번 같은 상태로 시작하도록** 만든다.
지금은 그것이 계약이 아니라 우연에 기대고 있다.

작성 2026-08-24 · 대상 `Assets/Scripts/Controllers/Monster/**` · 선행 07_Prerequisites 2.1 · 후속 2단계(ResourceManager 이음새) · 3단계(스포너 소유권)

한 장 요약

Unity의 `Awake` · `Start` 는 **오브젝트가 새로 만들어질 때만** 돈다. 풀에서 꺼내 쓰는 몬스터는 이 둘을 건너뛴다. 그래서 "매 스폰마다 해야 하는 일"을 놓을 자리가 없고, 지금은 그 일이 `OnEnable` · `Start` · `SetStat` · `HandleDeadState` 네 군데에 흩어져 있다. **이 문서는 그 자리를 `OnSpawn` / `OnDespawn` 두 개로 만들고, 흩어진 것을 옮기는 작업을 정의한다.**

0. 왜 지금 이 작업인가

측정의 유효성 문제 — 성능 개선이 아니다

이 트랙의 목적은 5가지 몬스터 관리 방법을 **같은 기준으로 재는 것**이다. 그런데 지금 상태로 재면 숫자가 방법의 차이가 아니라 **재사용 버그의 차이**를 잰다.

재려는 것	지금 섞여 들어오는 것	결과
히칭 프레임	풀 재사용 때 HP바를 다시 비동기 로드	방법마다 스폰 빈도가 달라 **순위가 뒤집힐 수 있다**
평균 활성 수	디스폰돼도 남는 HP바·구독	"껐다"의 정의가 흔들린다
스폰 1회 비용	초기화가 4군데로 흩어져 무엇을 뺐는지 말할 수 없다	개선의 원인을 특정할 수 없다

판단 기준 (07_PREREQUISITES 0절)

이걸 고치면 5가지 방법의 상대 순위가 바뀌나? → 바뀐다. 그래서 재기 **전에** 한다.

1. 이 문서의 범위

무엇을 하고 무엇을 안 하는지 먼저 못 박는다

한다

- 생애 훅 4개 정의 (`Awake` / `OnSpawn` / `OnEnable` / `OnDespawn`)
- 몬스터 컴포넌트의 **모든 필드가 어느 훅에서 리셋되는지** 전수 정리
- 구독·생성한 것을 같은 층에서 반납하도록 짝 맞추기
- 자식 컨트롤러(Normal/Ranged/AR/RL/Tank/Boss)까지 내려 적용

안 한다 (다음 단계)

- `ResourceManager` 에 `activate:false` 오버로드 추가 → **2단계**
- 풀 용량 외부 주입, `Pop` 즉석생성 경고 → **2단계**
- 스포너가 리스폰 상태를 소유 → **3단계**
- 스포너 배치 리팩터(앵커+패턴) → 실험 종료 후

중요 — 1단계만으로는 증상이 안 사라진다

`OnSpawn` 을 만들어도 **그것을 불러줄 사람이 2단계에 있다.** 지금은 `ResourceManager.Instantiate` 가 마지막에 `SetActive(true)` 를 해버려서 팩토리가 끼어들 틈이 없다. 1단계는 **받을 그릇을 만드는 일**이고, 물을 붓는 건 2단계다. **1단계 끝나고 플레이 테스트해서 "안 고쳐졌네" 하지 말 것.** 검증은 2단계 이후에 한다.

2. 개념 — 풀링이 무엇을 바꾸는가

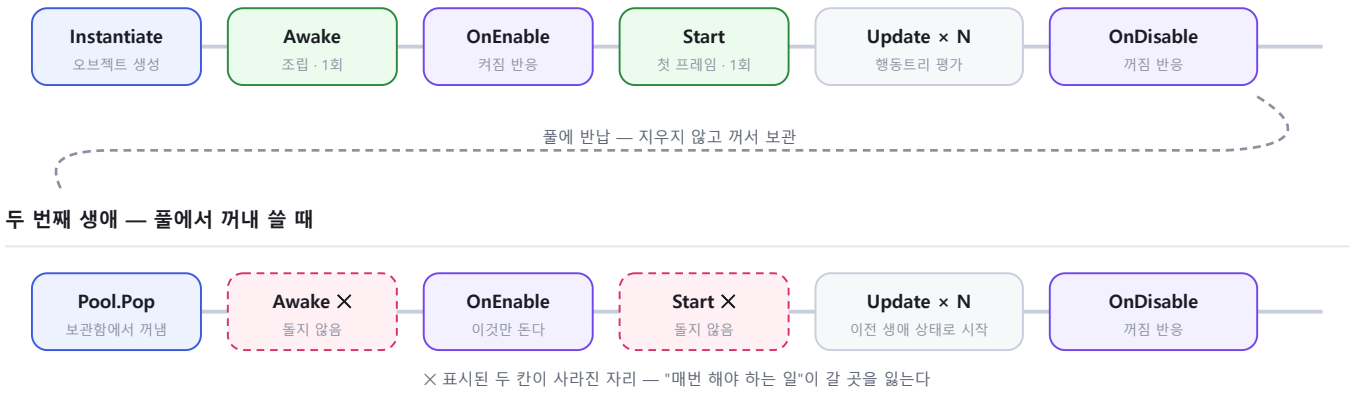
모든 문제가 여기 한 곳에서 나온다

Unity의 생명주기 함수는 두 종류로 나뉜다.

- **오브젝트가 태어날 때 딱 한 번** — `Awake`, `Start`
- **켜지고 꺼질 때마다** — `OnEnable`, `OnDisable`

오브젝트 풀링은 **죽은 오브젝트를 지우지 않고 꺼서 보관했다가 다시 켜는 것**이다. 그래서 **두 번째 생애부터는 `Awake`와 `Start`가 돌지 않는다.** 아래 그림이 이 명세서 전체의 출발점이다.

그림 1. 같은 오브젝트의 두 생애 — 무엇이 빠지는가 핵심



Awake·Start는 인스턴스당 1회. 풀 재사용은 인스턴스를 재활용하므로 이 둘을 건너뛴다.

그래서 실제로 무슨 일이 벌어지나

초기화 코드를 놓을 자리가 `OnEnable`밖에 없어진다. 그런데 `OnEnable`에는 두 가지 구조적 한계가 있다.

한계	내용
인자를 못 받는다	유니티가 부르는 함수라 `OnEnable(스탯, 레벨, 스폰지점)` 같은 형태가 불가능하다. 그래서 데이터는 **따로, 나중에** 넣게 되고 순서가 어긋난다.
이미 켜진 뒤다	`OnEnable`이 도는 순간 오브젝트는 이미 활성이다. 같은 프레임에 `Update`·물리·애니메이터가 돌기 시작한다. **준비가 끝나기 전에 게임이 시작된다.**

3. 실무 패턴 — 왜 다들 "스폰 훅"을 따로 만드나

이건 내가 지어낸 구조가 아니다

2장의 한계는 유니티만의 문제가 아니다. **재사용되는 오브젝트를 다루는 프레임워크는 예외 없이 "생성 훅"과 "스폰 훅"을 분리해 두었다.** 이름만 다를 뿐 같은 결론이다.

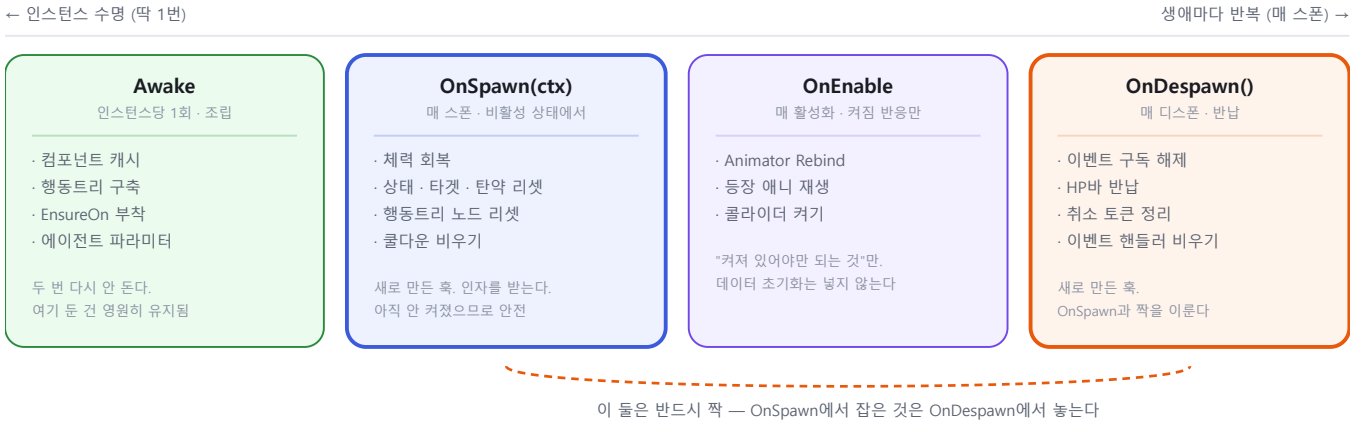
프레임워크	생성 / 파괴 훅	스폰 / 디스폰 훅
Unity MonoBehaviour	`Awake / OnDestroy`	**없음** — 직접 만들어야 한다
Unity ObjectPool<T>	`createFunc / actionOnDestroy`	`actionOnGet / actionOnRelease`
Netcode for GameObjects	`Awake / OnDestroy`	`OnNetworkSpawn / OnNetworkDespawn`
Photon Fusion	`Awake / OnDestroy`	`Spawned() / Despawned()`
Unreal Actor	생성자 / `BeginDestroy`	`BeginPlay / EndPlay`

읽는 법

표의 오른쪽 열이 비어 있는 건 **MonoBehaviour 하나뿐**이다. 유니티가 기본으로 안 주니까 프로젝트마다 직접 만든다 — 그게 이번 작업이다. **"풀링을 쓰면서 스폰 훅을 안 만든 것"이 결함이지, 만드는 게 특별한 설계가 아니다.**

3.1 채택할 계약 — 훅 4개

그림 2. 생애 훅 4개와 각자의 성격 계약



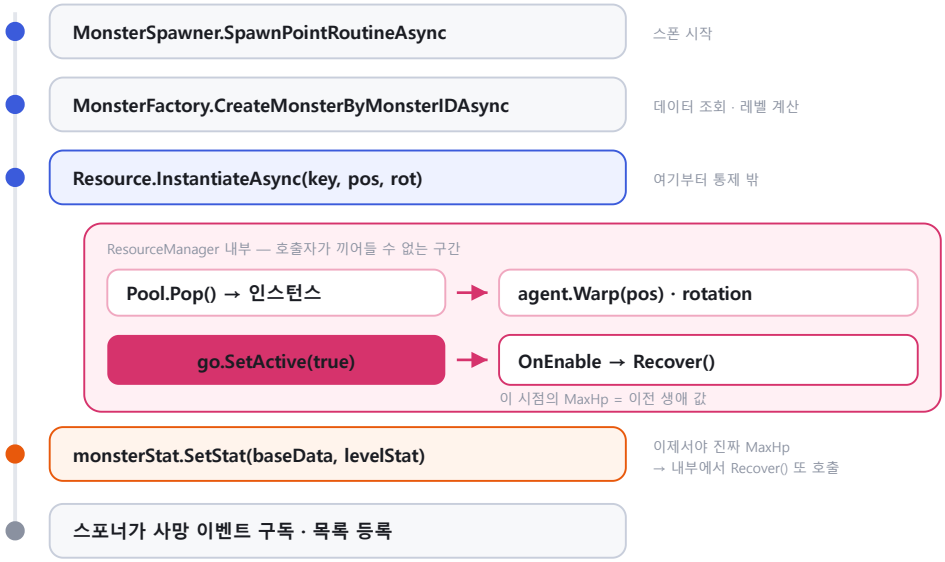
왼쪽 두 칸이 "준비", 오른쪽 두 칸이 "반응과 반납". 파랑·주황이 새로 생기는 계약이다.

4. 진단 ① — 지금의 스폰 순서

실제 코드를 따라간 호출 순서

스포너가 몬스터 하나를 만들 때 실제로 일어나는 일이다. **위에서 아래로 따라가면 된다.**

그림 3. 현재 스폰 경로 — 체력을 먼저 채우고 최대체력을 나중에 넣는다 문제



붉은 상자가 문제 구간. `SetActive(true)`가 ResourceManager 안쪽에 있어서 팩토리가 그 앞에 못 들어간다.

이게 왜 "지금 안 터지는데?"인가 — 가장 중요한 대목

`Recover()`가 **두 번** 불린다. 첫 번째는 `OnEnable`에서 **이전 생애의 MaxHp**로, 두 번째는 `SetStat` 끝에서 **올바른 MaxHp**로. **두 번째가 첫 번째를 덮어써서 최종 체력은 맞는다.**

즉 지금 동작하는 이유는 설계가 아니라 **덮어쓰기가 우연히 순서를 맞춰주기 때문**이다. `SetStat`에서 `Recover()` 한 줄만 지우면 그날로 깨진다. 이것이 **"우연히 맞는 코드(incidental correctness)"**이고, 감사 등급으로는 **잠재 결함**이다.

4.1 그 사이에 이미 벌어지는 일들

최종 체력은 맞더라도, `SetActive(true)`와 `SetStat` 사이에 아래가 이미 실행된다.

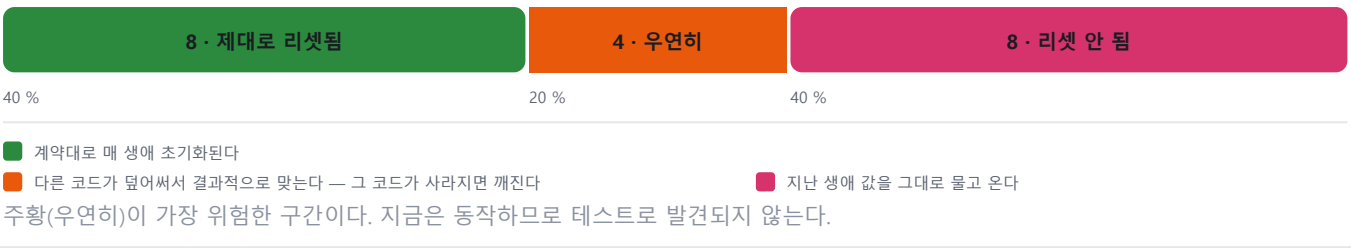
그 사이에 시작되는 것	무엇을 보고 시작하나
HP바 비동기 로드 NormalMonsterController.OnEnable	이전 생애 스탯. 로드가 끝나 `SetTarget`이 불릴 땐 값이 맞지만, **시작 시점의 근거가 잘못돼 있다**
에이전트 지연 활성 RangedMonsterController.OnEnable	1프레임 뒤 `_agent.enabled = true`. 이때 위치가 `Warp`로 들어간 값인지 보장이 없다
행동트리 평가 BaseMonsterController.Update	`CheckIsDead`가 `CurrentHp`를 읽는다. 스탯 주입 전 한 프레임을 이전 값으로 돈다
이벤트 발화 Health.OnHpChanged	`Recover`가 `OnHpChanged`를 쏜다 → 한 프레임에 **두 번** 발화

5. 진단 ② — 상태 인벤토리

몬스터가 들고 있는 모든 값이 언제 리셋되는지 전수 조사

실무에서 **pooled object state audit**이라고 부르는 작업이다. "재사용될 때 이 값이 지난 생애 것을 물고 오는가?"를 필드 하나하나에 묻는다. 아래는 몬스터 컴포넌트 전체를 훑은 결과다.

그림 4. 리셋 커버리지 — 20개 상태 중 절반이 계약 밖 진단



주황(우연히)이 가장 위험한 구간이다. 지금은 동작하므로 테스트로 발견되지 않는다.

클래스	상태	판정	지금 어디서 리셋되나 / 안 되면 무슨 일
BaseMonster	`_state`	OK	`OnEnable`에서 `Spawning`
	`_monsterCts`	OK	`OnEnable`에서 재생성, `OnDisable`에서 취소
	`_anim / _col`	OK	`Rebind` · `enabled = true`
	`OnDespawned`	우연	`OnEnable`에서 `null` — **다음 생애가 시작될 때 지난 생애 흔적을 지운다.** 순서가 거꾸로다
	`OnDead`	없음	아무도 비우지 않는다. 구독자가 쌓이면 중복 발화
	`_abilityRunner._cd`	없음	쿨다운이 **절대 시각**으로 기록된다. 죽고 7초 만에 리스폰되면 쿨 10초짜리는 **쿨 상태로 부활**
NormalMonster	`_target`	없음	`Start`에서만 설정 → 재사용 시 지난 생애 타겟 유지
	`_hpBar`	우연	**죽었을 때만** 반납된다. 섹터 이탈 디스폰에는 정리 코드가 없다 (→ 6장)
	`_lastDestPosition`	없음	지난 목적지가 남아 **부활 직후 첫 `SetDestination`이 스킵될 수 있다**
	`_agent.enabled`	우연	사망 경로에서만 `false`. 디스폰 경로는 켜진 채 반납 → `Warp` 신뢰성 문제
	`_initialized`	OK	인스턴스 수명 플래그 — **의도적으로** 안 지운다
RangedMonster	`_currentAmmo`	OK	`OnEnable`에서 `_maxAmmo`
MonsterTank	`_flag`	없음	착탄 오프셋 순번이 지난 생애에서 이어진다
MonsterStat	`OnMonsterDead`	우연	`SetStat`이 `null`로 비운다 — **이벤트 소유자가 아닌 쪽이 남의 구독을 지운다** (감사 M8)
	`OnKilledByAttacker`	OK	`MonsterReward.Bind`가 `-=` 후 `+=` (멱등)
	`HealthComp`	OK	`SetStat` 끝의 `Recover()`

BTNode 3종	`RandomNode._currentNode`
	`WaitNode._isWaiting`
	`BossSequence._childIndex`
		없음	노드가 상태를 들고 있는데 리셋 지점이 없다. **지금은 일반몹 트리가 이 노드를 안 쓰고 보스는 풀을 안 타서 발현하지 않는다** (감사 M5 · 잠재)

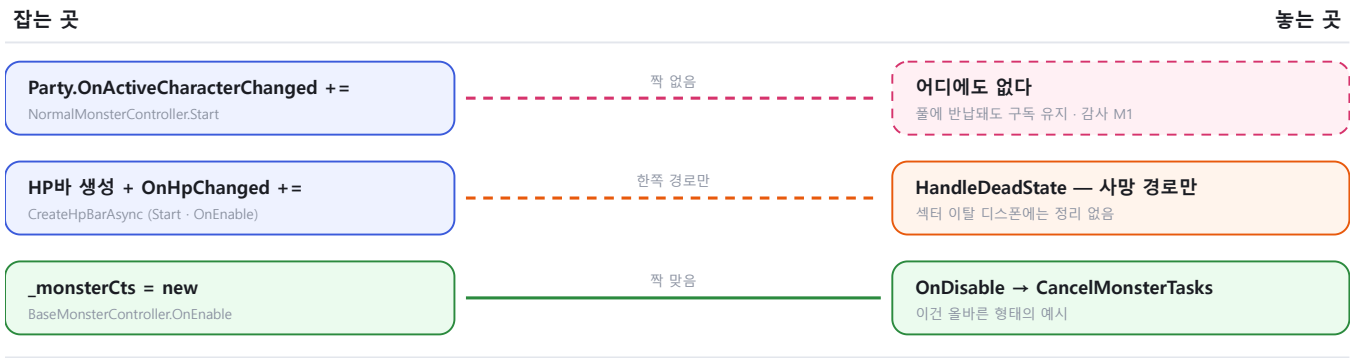
6. 진단 ③ — 짝이 안 맞는 것들

잡은 곳과 놓는 곳이 다른 층에 있다

프로그래밍에서 가장 오래된 규칙 중 하나가 **"잡은 사람이 놓는다"**이다. 파일을 열었으면 닫고, 메모리를 할당했으면 해제하고, **이벤트를 구독했으면 해제한다.** C++의 RAII, C#의 `using`, 유니티의 `OnEnable` / `OnDisable` 쌍이 전부 같은 발상이다.

지금 몬스터는 이 짝이 세 군데에서 깨져 있다.

그림 5. 획득과 반납의 대칭 — 점선은 끊어진 짝 문제



세 번째 줄이 정답의 모양이다 — 같은 층(OnEnable ↔ OnDisable)에서 잡고 놓는다.
위 두 줄은 잡는 층과 놓는 층이 달라서, 경로에 따라 놓치는 경우가 생긴다.

6.1 대표 증상 — 몬스터 없는 HP바

두 번째 줄이 만드는 실제 버그를 따라가 보자.

순서	일어나는 일
1	플레이어가 섹터를 벗어난다 → `DespawnMonsters()` → `Resource.Destroy(monster)` → **풀에 반납(비활성화)**
2	몬스터는 꺼졌다. 그런데 HP바는 `CanvasWorld` 자식이라 **여전히 켜져 있다**
3	`UI_MonsterHPBar.LateUpdate`는 `_targetTr == null`일 때만 자기를 지운다. **비활성 오브젝트는 `null`이 아니다** → 안 지워진다
4	결과 — **몬스터 없는 HP바가 남아 매 프레임 `LateUpdate`를 돈다**
5	다시 그 섹터에 들어가도 `_hpBar ≠ null`이라 재생성 조건에 안 걸린다. 옛 바를 계속 쓴다

이게 측정에 미치는 영향

방법4(거리 기반)는 **밴드를 넘나들 때마다 디스폰**한다. 즉 **이 경로를 가장 많이 타는 방법이다.** 고치지 않고 재면 방법4에만 유령 HP바가 쌓이고, 그 `LateUpdate` 비용이 "거리 기반 방식이 비싸다"는 결론으로 잘못 읽힌다. **순위를 바꾸는 오염이므로 재기 전에 고쳐야 한다.**

7. 설계 — 목표 구조

순서를 뒤집고, 훅을 두 개 만든다

그림 6. 목표 스폰 경로 — 다 준비하고 나서 켠다 목표

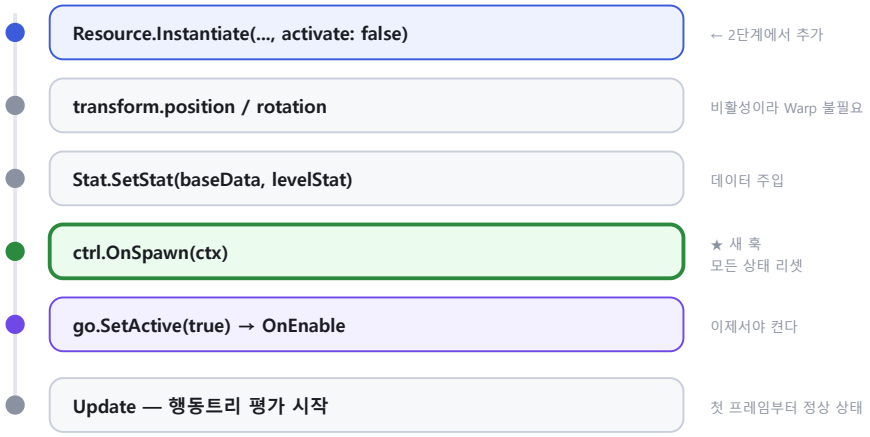


그림 3과 비교하면 `SetStat`과 `SetActive`의 순서가 뒤집혔다. 이 한 번의 뒤집기가 4장의 문제 4개를 모두 없앤다.

7.1 훅 배치표 — 무엇을 어디로 옮기나

훅	담을 것	지금 어디에 있나
Awake	컴포넌트 캐시 · 행동트리 구축 · `MonsterReward` / `DamageNumberPresenter` 부착 · 에이전트 파라미터	**그대로 유지**
OnSpawn(ctx)	· `_state = Spawning` · `_monsterCts` 재생성 · `OnDead` / `OnDespawned` 비우기 · `_currentAmmo` · `_flag` · `_lastDestPosition` · `_target` 리셋 · 행동트리 노드 리셋 (`_topNode.Reset()` 신설) · `_abilityRunner` 쿨다운 비우기 · `HealthComp.Recover()`	흩어져 있음 — `OnEnable` (Base · Ranged) `Start` (Normal) `SetStat` (Stat)
OnEnable	· `_col.enabled = true` · `_anim.Rebind()` + `Play("Appear")` · 에이전트 지연 활성	같은 자리 — **데이터 초기화만 빠져나간다**
OnDespawn()	· `Party.OnActiveCharacterChanged -=` · HP바 구독 해제 + 반납 · `Stat.OnMonsterDead` 구독 해제 · `CancelMonsterTasks()` · `_agent.enabled = false`	일부만 `HandleDeadState` (사망 경로)에 나머지 **없음**

설계 포인트 — ONDESPAWN은 두 경로가 함께 부른다

지금은 **사망**과 **디스폰**이 서로 다른 정리 코드를 갖고 있어서 한쪽만 챙겨진다. `OnDespawn()`을 만들면 두 경로가 **같은 정리 함수**로 합류한다.

사망 → 연출 2초 → `OnDespawn` → 풀 반납 · 섹터 이탈 → `OnDespawn` → 풀 반납

8. 설계 원칙 4개

이 문서의 모든 판단이 이 넷에서 나온다

P1. 조립과 초기화를 나눈다 construction vs initialization

조립은 "이 오브젝트가 무엇으로 이루어졌나" — 컴포넌트 찾기, 트리 만들기. 한 번만 하면 된다.
초기화는 "이번 생애를 어떤 값으로 시작하나" — 체력, 탄약, 타겟. 매번 해야 한다.
지금은 이 둘이 `Awake` · `OnEnable` · `Start` 에 섞여 있다. 섞이면 **어느 것이 매번 도는지 코드만 봐서는 알 수 없다.**

P2. 데이터를 먼저 넣고 나중에 켠다 initialize before activate

활성화는 **돌이킬 수 없는 사건**이다. 켜는 순간 `Update`·물리·애니메이터·코루틴이 전부 출발한다. 그 전에 모든 값이 제자리에 있어야 한다.
부수 효과 — 비활성 상태에서 위치를 옮기면 `NavMeshAgent.Warp`가 필요 없어진다. 에이전트는 켜질 때 자기 위치를 기준으로 NavMesh에 스냅하기 때문이다. 지금은 꺼진 에이전트에 `Warp`를 부르고 있어 위치가 반영 안 될 수 있다 — **2단계에서 검증 대상.**

P3. 잡은 층에서 놓는다 acquire / release symmetry

`OnSpawn`에서 구독했으면 `OnDespawn`에서 해제한다. `OnEnable`에서 잡았으면 `OnDisable`에서 놓는다. **층을 건너뛰면 경로에 따라 새는 구멍이 생긴다** — 6장의 HP바가 정확히 그 사례다.
같은 이유로 **이벤트를 비우는 주체는 그 이벤트의 소유자**여야 한다. `MonsterStat.SetStat`이 `OnMonsterDead = null`을 하는 건 **스포너가 건 구독을 스탯이 지우는** 형태라 잘못됐다.

P4. 우연히 맞는 것을 계약으로 바꾼다 incidental correctness → contract

지금 체력이 맞는 이유는 `Recover()`가 두 번 불리기 때문이고, 탄약이 맞는 이유는 `OnEnable`에 마침 그 줄이 있기 때문이다. **둘 다 "이렇게 하기로 정해서"가 아니라 "어쩌다 보니"다.**
이런 코드는 테스트로 발견되지 않는다. **지금은 동작하니까.** 그래서 상태 인벤토리(5장) 같은 전수 조사가 필요하고, 표에 **주황(우연히)** 칸을 따로 둔 것이다.

9. 작업 순서

파일 단위 체크리스트

#	파일	할 일
1	`IPooledMonster` 신설	`OnSpawn(in SpawnContext)` / `OnDespawn()` 인터페이스 정의. 컨텍스트에 담을 것 = 스탯 데이터 · 스폰 지점 · 소유 스포너
2	`BaseMonsterController`	훅 2개 구현 · `OnEnable` 에서 데이터 초기화 걷어내기 · `OnDead` / `OnDespawned` 비우기를 `OnDespawn` 으로 · 쿨다운 초기화
3	`BTNode`	추상 `Reset()` 추가. `Selector` / `Sequence` 는 자식 전파, 상태 노드 3종은 자기 필드 리셋
4	`NormalMonsterController`	**핵심** `Start` 의 파티 구독 → `OnSpawn` , 해제를 `OnDespawn` 에 · HP바 반납을 `OnDespawn` 으로 일원화 · `_target` / `_lastDestPosition` 리셋 · `_initialized` 유지 여부 판단
5	`RangedMonsterController`	`_currentAmmo` 를 `OnEnable` → `OnSpawn` 으로 이동
6	`MonsterTankController`	`_flag` 리셋 추가
7	`MonsterStat`	`OnMonsterDead = null` 제거(소유자에게 반환) · `Recover()` 호출 유지 여부 결정
8	`BossMonsterController`	연출·사망 플래그를 `OnSpawn` 으로 (감사 M6). 보스는 현재 풀을 타지 않으므로 **계약만 맞추고 검증은 생략**

10. 완료 판정

무엇이 참이면 이 단계가 끝났다고 말할 수 있나

1단계만으로는 눈에 보이는 변화가 없다(1장 참고). 그래서 **완료 판정은 동작이 아니라 성질로 한다.**

#	확인할 것	방법
C1	5장 표의 **빨강 8개**가 전부 **초록**이 된다	표를 그대로 체크리스트로 재사용. 각 필드가 `OnSpawn` / `OnDespawn` 중 하나에 명시적으로 등장하는지 확인
C2	**주황 4개**도 초록이 된다	"덮어써서 맞는" 형태를 없앤다. `SetStat` 의 `Recover()` 를 지워도 체력이 맞아야 한다
C3	구독·해제가 같은 층에 있다	`+=` 를 전수 검색해 각각의 `-=` 가 짝 훅에 있는지 대조 (그림 5)
C4	디스폰 경로가 하나로 합류한다	사망 경로와 섹터 이탈 경로가 **같은** `OnDespawn()` 을 통과하는지
C5	유령 HP바가 0이다	섹터 왕복 10회 후 `CanvasWorld` 자식 수가 진입 전과 같은지 (**2단계 이후 실측**)

C5만 뒤로 미루는 이유

C1~C4는 **코드를 읽어서 판정**할 수 있다. C5는 실제로 돌려봐야 하는데, 그러려면 `OnSpawn` 을 불러줄 호출자(2단계)가 있어야 한다. **1단계 완료 판정은 C1~C4로 하고, C5는 2단계 완료 판정에 넘긴다.**

11. 남기는 기록

이 작업이 포트폴리오에서 하는 역할

이 단계는 성능 수치를 만들지 않는다. 대신 **세 가지를 기록으로 남긴다.**

① 상태 인벤토리 표 (5장)
"풀링을 쓴다"고 말하는 것과 **"풀링 때문에 무엇이 깨지는지 전수 조사했다"**고 말하는 것은 다르다. 20개 중 12개가 계약 밖이었다는 숫자가 그 차이를 만든다.

② "우연히 맞는 코드"의 발견 (4장)
체력이 맞는 이유가 `Recover()` 두 번 호출이었다는 것. **버그를 고친 이야기보다, 안 터지는 이유를 설명할 수 있는 편이 강하다.**

③ 측정 오염원의 제거 근거 (6.1)
유령 HP바는 **방법4에만 많이 낀다.** 이걸 안 고치고 재면 "거리 기반 방식이 비싸다"는 잘못된 결론이 나온다. **측정 전에 통제 변수를 확보했다는 것** 자체가 이 트랙의 주장 중 하나다.

면접에서 이 단계를 한 줄로 말한다면
"오브젝트 풀링을 쓰면 `Awake`와 `Start`가 안 도니까, **매 스폰마다 해야 하는 일을 놓을 자리를 따로 만들었습니다.** 그러면서 몬스터가 들고 있는 상태 20개를 전부 훑었는데 절반이 지난 생애 값을 물고 오고 있었습니다. 그중 **지금 안 터지는 것들이 더 위험해서** 따로 표시해 뒀습니다."

12. 온라인 게임이었다면 — 무엇이 달라지나

상위 주제(대규모 필드 엔티티 관리)와의 연결

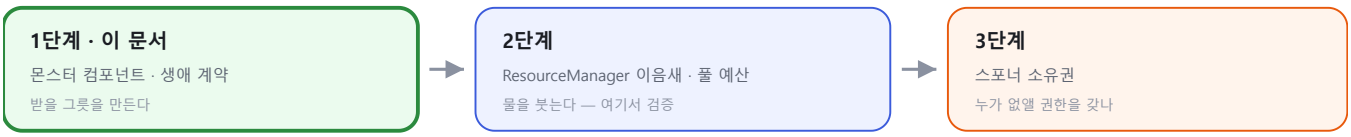
이 프로젝트에는 서버가 없다. 그래서 **존재(누가 있나)와 품질(얼마나 잘 도나)을 클라가 전부 갖는다.** 서버가 생기면 이 명세서의 어디가 이사 가고 어디가 남는지 미리 정리해 둔다.

이 문서의 항목	서버가 생기면	근거
누구를 스폰할지 결정	**서버로 이동**	클라가 정하면 맵핵이 된다. 존재는 신뢰 경계 안쪽에 있어야 한다
OnSpawn / OnDespawn 훅	**그대로 남음**	이름이 `OnNetworkSpawn` / `Spawned()`로 바뀔 뿐 **같은 자리, 같은 역할** (3장 표)
상태 인벤토리	**그대로 남음**	오히려 더 중요해진다. 클라 프록시가 지난 생애 값을 물고 오면 **서버와 어긋난 화면**이 된다
HP바 수명 관리	**그대로 남음**	100% 클라 표현 계층
스폰 1회 프레임 비용	**그대로 남음**	대역폭이 아니라 프레임 예산 문제 — 서버가 대신 내주지 않는다

정리
5개 중 **서버로 넘어가는 건 하나뿐**이다. 나머지 넷은 온라인 게임의 클라이언트 개발자 책상에도 그대로 있다.
싱글 게임은 온라인의 쉬운 버전이 아니라 관찰자가 한 명인 버전이고, 관찰자가 한 명이라 실제로 만들고 잴 수 있다.

13. 이 다음

3단계까지의 흐름



그 뒤 → 측정 하네스 → 기준선(방법1·2) → 방법3·4·5 → 전 조합 측정 → 실제 맵 적용

상세는 07_Prerequisites 4절

착수 전 마지막 확인 — 복구 불가능한 두 가지

① **예측 박제** — 방법4를 만들기 시작하면 결과가 보인다. 그 뒤에 쓴 예측은 예측이 아니다.
② **경계 소실 Before 영상** — 방법3을 만들면 다시 찍을 수 없다. 방법2의 유일한 시각 증거다.
둘 다 합쳐서 반나절이고, **코드를 손대는 순간부터 값이 떨어지기 시작한다.**